Projet MLSF : résolution du jeu Meowdoku

Question 2 : lecture DIMACS et algorithme DPLL

1 Objectif et organisation

La question demande de lire un fichier DIMACS dans une structure de données Python, d'implémenter les algorithmes 1 et 2 de l'énoncé, de les valider puis de commenter leurs performances. Le code se répartit en deux fichiers :

- `dimacs.py` : lecture d'un fichier DIMACS et vérification qu'une valuation satisfait une formule ;
- `dpll.py` : algorithme 1 (`dpll_sat`), algorithme 2 (`dpll`) et fonctions auxiliaires.

Les tests se trouvent dans le dossier `tests` et s'exécutent avec :

```
uv run python -m unittest discover -s tests -t .
```

Comme pour la question 1, seule la bibliothèque standard de Python 3.12 est utilisée : c'est la version imposée pour la compétition de la question 6.

2 Représentation des formules et lecture DIMACS

2.1 Structure de données

Une formule CNF est représentée par un couple `(nb_vars, clauses)`. La liste `clauses` contient une clause par élément ; chaque clause est un `frozenset` d'entiers. Nous conservons la convention DIMACS : l'entier k représente x_k et l'entier $-k$ représente $\neg x_k$. Par exemple, l'énigme des trois lampes de l'énoncé devient :

```
(3, [frozenset({1, 2, 3}), frozenset({-1, -2}),
     frozenset({-2, -3}), frozenset({2, 3})])
```

Ce choix est guidé par l'opération centrale de DPLL, la simplification $F[l \leftarrow 1]$:

- **Pas de traduction.** Les littéraux du fichier sont utilisés tels quels ; la négation d'un littéral `l` est simplement `-l` et sa variable `abs(l)`.
- **Tests d'appartenance en temps constant.** Savoir si une clause contient l ou $\neg l$ est un test `in` sur un ensemble.
- **Doublons éliminés.** Une clause `1 1 -2 0` devient $\{1, -2\}$, ce qui correspond à la sémantique d'une clause, vue comme un ensemble de littéraux.
- **Immuabilité.** Un `frozenset` ne peut pas être modifié. Chaque appel récursif travaille sur une nouvelle formule et le retour arrière consiste simplement à revenir à la formule de l'appel parent, qui n'a pas été touchée. C'est exactement le modèle des algorithmes de l'énoncé, où $F[p \leftarrow b]$ et $F[p \leftarrow \overline{b}]$ sont deux formules indépendantes.

Alternatives écartées. Une liste de listes est plus compacte, mais retirer un littéral d'une clause y coûte un parcours de la clause et les doublons doivent être gérés à part. Une structure mutable, avec annulation des modifications lors du retour arrière (pile de modifications, littéraux surveillés), est bien plus rapide : elle évite de recopier la formule à chaque appel. Elle est en revanche plus délicate à écrire et à justifier. Nous la réservons à la version optimisée des questions 4 à 6 ; la version de la question 2 sert de référence simple et sûre.

2.2 Règles de lecture

La fonction `parse_dimacs` lit le texte ligne par ligne :
— les lignes vides et les lignes commençant par `c` sont ignorées ;
— l'en-tête `p cnf V C` est obligatoire, unique, et doit précéder toute clause ;
— une clause est une suite d'entiers terminée par `0`. Elle peut s'étendre sur plusieurs lignes, et plusieurs clauses peuvent partager une ligne : nous lisons un flux d'entiers plutôt qu'une clause par ligne ;
— une ligne `%` arrête la lecture. Les fichiers de la bibliothèque SATLIB se terminent par `%` puis `0`, qui serait sinon lu comme une clause vide et rendrait la formule insatisfiable à tort.

La fonction `read_dimacs` applique `parse_dimacs` au contenu d'un fichier. Séparer les deux permet de tester la lecture sur des chaînes de caractères, sans créer de fichiers.

2.3 Erreurs et tolérances

Comme pour `read_meowdoku`, une entrée mal formée provoque une `ValueError` avec le numéro de la ligne fautive : en-tête absent, répété ou mal formé, littéral non entier, ou variable hors de $\{1, \dots, V\}$. Ce dernier contrôle est nécessaire, car le modèle affiché à la question 6 porte sur les variables 1 à V.

Deux écarts au format sont en revanche tolérés, car ils ne rendent pas la formule ambiguë :
— le nombre de clauses C de l'en-tête n'est pas vérifié ; il n'est pas nécessaire à la résolution ;
— une dernière clause sans `0` final est conservée.

Ce choix a été fait avant de disposer des fichiers des enseignants : mieux vaut qu'un fichier légèrement négligé soit lu correctement que rejeté, surtout lors de la compétition où une instance non lue est une instance perdue.

Deux cas particuliers sont conservés tels quels. Une clause vide (`0` seul) est gardée : DPLL répond alors immédiatement UNSAT, ce qui est la bonne réponse. Une clause tautologique comme $x_1 \vee \neg x_1$ est également gardée : elle ne gêne pas DPLL et disparaît dès que x_1 est affectée. La supprimer dès la lecture est une optimisation possible pour la suite.

2.4 Vérification d'un modèle

La fonction `check_model(clauses, model)` renvoie vrai si chaque clause contient un littéral rendu vrai par la valuation `model`, un dictionnaire {variable : booléen}. Un littéral dont la variable n'est pas affectée n'est *pas* considéré comme vrai. Cette définition stricte est celle qu'appliquera le correcteur automatique de la compétition ; elle oblige le solveur à fournir une valuation complète (voir section 3.5). La fonction est indépendante du solveur, ce qui en fait un juge impartial dans les tests.

3 Implémentation de DPLL

3.1 Simplification

La fonction `simplify(clauses, l)` calcule $F[l \leftarrow 1]$ en un seul parcours, conformément à la définition de l'énoncé : une clause contenant l est vraie et disparaît ; une clause contenant $\neg l$ perd ce littéral et peut devenir vide ; les autres clauses sont reprises sans copie. Affecter la variable p à la valeur b revient à affecter le littéral p (si $b = 1$) ou $-p$ (si $b = 0$) à vrai : une seule fonction suffit donc pour les règles unitaire, pure et de branchement.

3.2 Correspondance avec les algorithmes de l'énoncé

Les deux algorithmes appliquent les règles dans l'ordre de l'énoncé :

1. clause vide : UNSAT ;
2. formule sans clause : SAT ;
3. clause unitaire $\{l\}$: on continue avec $F[l \leftarrow 1]$;
4. littéral pur l : on continue avec $F[l \leftarrow 1]$;
5. sinon, choix aléatoire (p, b), essai de $F[p \leftarrow b]$, puis en cas d'échec de $F[p \leftarrow \overline{b}]$.

Les étapes 1 à 4 sont communes aux deux versions et regroupées dans `_propagate`. `dpll_sat` (algorithme 1) renvoie un booléen ; `dpll` (algorithme 2) renvoie `(SAT, v)` ou `(UNSAT, None)`. Nous utilisons `None` plutôt que le 0 de l'énoncé : c'est la convention Python pour « pas de valeur » et elle ne peut pas être confondue avec une valuation.

Un littéral pur est cherché en rassemblant tous les littéraux de la formule dans un ensemble, puis en prenant le premier l (par numéro de variable croissant) tel que $\neg l$ est absent. Le tri rend le choix indépendant de l'ordre interne de l'ensemble, donc reproductible.

3.3 Propagation par une boucle plutôt que par récursion

Dans l'énoncé, les règles unitaire et pure sont des appels récursifs : chaque clause unitaire ajoute un niveau de récursion. Or Python limite par défaut la profondeur de récursion à 1000 appels. Une formule qui propage plusieurs centaines de clauses unitaires, ce qui est courant (une grille Meowdoku de taille n a n^2 variables), ferait échouer le programme.

Nous remplaçons donc ces deux appels par une boucle. La transformation ne change pas le résultat : dans les lignes 5 à 16 de l'algorithme 2, l'appel récursif est le dernier calcul effectué. Son statut est renvoyé tel quel et la valuation est seulement complétée par $\{l\}$. Il suffit donc de mémoriser les littéraux fixés dans une liste (`forced`) et de les ajouter à la valuation une fois le statut connu.

Seul le branchement reste récursif, car il nécessite un vrai retour arrière. La profondeur de récursion est alors au plus le nombre de décisions d'une branche, donc au plus le nombre de variables : chaque décision fait disparaître sa variable. Par sécurité, la limite de récursion est relevée au nombre de variables plus 1000 si nécessaire. Depuis Python 3.11, les appels entre fonctions Python ne consomment plus la pile du langage C, ce qui rend cette augmentation sans risque. Un test vérifie une formule demandant 3000 décisions imbriquées.

3.4 Choix aléatoire et reproductibilité

La fonction `random_choice` correspond à `RandomChoice` : elle tire uniformément une variable parmi celles qui apparaissent encore dans la formule, puis une valeur de vérité. Le couple (p, b) est renvoyé sous la forme d'un littéral (p ou $-p$) ; la branche « sinon » utilise son opposé. Isoler ce choix dans une fonction prépare la question 4, où il sera remplacé par `Heuristics(F, H)`.

Le générateur aléatoire est passé en paramètre (`rng`, un objet `random.Random`). Avec une graine fixée, une exécution est donc reproductible, ce qui est indispensable pour déboguer et pour comparer les mesures de performance. Les variables sont triées avant le tirage pour que le résultat ne dépende que de la graine.

3.5 Valuation partielle et complétion

La valuation renvoyée par l'algorithme 2 est partielle. Une variable peut disparaître de la formule sans être affectée : c'est le cas lorsque toutes les clauses qui la contiennent ont été satisfaites par d'autres littéraux. Par exemple, pour la formule $(x_1 \vee x_2)$, le littéral x_1 est pur : la règle fixe x_1 à vrai, la clause disparaît et la formule est vide. La valuation renvoyée est $\{x_1 \leftarrow 1\}$ et x_2 n'y figure pas.

Toute valeur convient alors pour ces variables : au moment où elles disparaissent, chaque clause qui les contient est déjà satisfaite par un littéral affecté. La fonction `complete` leur

donne la valeur faux et produit une valuation sur toutes les variables $1, \ldots, V$, ce qu'exigent `check_model` et le format de sortie de la question 6.

3.6 Compteurs

Une structure `Stats` compte les décisions, les retours arrière, les propagations unitaires et les applications de la règle du littéral pur. Ces compteurs correspondent aux critères de la question 5. Leur coût est négligeable (une incrémentation par étape) et ils permettent dès maintenant d'expliquer les temps observés.

3.7 Complexité

Notons $|F|$ la taille de la formule, c'est-à-dire le nombre total de littéraux. Chaque nœud de l'arbre de recherche coûte $O(|F|)$ pour `simplify`, la recherche de clause vide et de clause unitaire, et $O(|F| + v \log v)$ pour la recherche de littéral pur et le choix aléatoire, avec v le nombre de variables restantes. Une propagation de u littéraux coûte donc $O(u\,|F|)$. Le nombre de nœuds est au pire exponentiel en le nombre de variables : DPLL reste exponentiel dans le pire cas, ce qui est attendu pour un problème NP-complet.

4 Validation

La validation repose sur plusieurs sources indépendantes : un oracle par force brute, des grilles Meowdoku, des cas limites, des benchmarks SATLIB et les instances publiques fournies par les enseignants.

Oracle par force brute. 1500 formules aléatoires de 1 à 10 variables, avec jusqu'à $5n$ clauses de 1 à 4 littéraux, sont résolues par les deux algorithmes et par une énumération des 2^n valuations. Le jeu contient plusieurs centaines d'instances de chaque statut, ce que le test vérifie. Pour chaque formule, les deux algorithmes doivent donner le statut de l'oracle, et toute valuation renvoyée, une fois complétée, doit satisfaire la formule d'après `check_model`.

Pour nous assurer que ce test détecte réellement les erreurs, nous avons volontairement introduit un défaut dans `simplify` (oubli de retirer $\neg l$ des clauses) : 979 des 1500 cas échouent alors.

Instances Meowdoku. Les grilles générées à la question 1 pour $n = 4$ à 9 sont satisfiables par construction ; le solveur doit répondre SAT avec un modèle valide. Trois grilles construites à la main doivent être insatisfiables : une grille 2×2 (deux chats forcément voisins), une grille 3×3 (aucune solution n'existe pour $n = 3$) et une grille 4×4 où deux régions d'une seule case sont adjacentes.

Cas limites. Formule vide (SAT), clause vide (UNSAT), $x \wedge \neg x$ (UNSAT), tautologie (SAT), exemple des trois lampes, chaîne de 3000 implications se terminant par une contradiction (uniquement de la propagation unitaire) et 3000 décisions imbriquées.

Benchmarks SATLIB. Avant de recevoir les fichiers des enseignants, nous avons utilisé les instances 3-SAT aléatoires de la bibliothèque de référence SATLIB [1], placées dans `data/satlib` par le script `download_satlib.py` (elles ne sont pas incluses dans le dépôt). Leur statut est connu par construction : les familles `uf` sont satisfiables, les familles `uuf` insatisfiables. Ces fichiers se terminent par la ligne `%` suivie d'un `0` : ils ont confirmé l'utilité de la règle d'arrêt du lecteur.

1. `https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html`

Le script `validate.py` parcourt un dossier et, pour chaque fichier, lance les deux algorithmes avec la même graine. Il vérifie que les deux statuts concordent, que le statut attendu (lu dans un fichier `expected.json` s'il existe, sinon déduit du nom : préfixe `uf` ou `uuf`, ou mention `sat` / `unsat`) est obtenu, et que le modèle complété satisfait la formule. Chaque algorithme est interrompu après 60 s, la limite de la compétition :

```
uv run python validate.py data/satlib/uf50-218
```

Famille	Variables	Clauses	Instances	Échecs	Temps total (algos 1 et 2)
`uf20-91`	20	91	1000	0	1,0 s
`uf50-218`	50	218	1000	0	52,9 s
`uuf50-218`	50	218	1000	0	147,7 s
`uf75-325`	75	325	100	0	104,3 s
`uuf75-325`	75	325	100	0	264,3 s

Les 3200 instances sont correctement résolues par les deux algorithmes, sans aucun dépassement du délai. Le temps indiqué est le total pour toute la famille : pour uuf75-325, 264,3 s pour 100 instances, soit environ 2,6 s par instance pour les deux algorithmes, bien en dessous de la limite de 60 s par instance.

Instances publiques des enseignants. Le dossier `public-instances` contient 41 fichiers et un fichier `expected.json` donnant le statut attendu de chacun : 11 petits tests d'interface et cas limites (formule vide, formule sans clause, clause vide, tautologie, littéraux répétés, littéraux tous négatifs), et 30 instances de benchmark issues de SATLIB (19 SAT, 11 UNSAT, de 42 à 1363 variables).

```
uv run python validate.py public-instances -v --timeout 60
```

Instance	Variables	Clauses	Attendu	Résultat (graine 0, algos 1 et 2)
`public-11`	729	21844	SAT	SAT en 29,8 s
`public-18`	459	7054	SAT	SAT en 9,9 s
`public-19`	729	22060	SAT	SAT en 10,3 s
`public-03`	63	168	UNSAT	timeout (60 s)
`public-12`	522	11636	SAT	timeout (60 s)
`public-27`	181	3151	SAT	timeout (60 s)
27 autres benchmarks				tous corrects, moins de 2 s chacun
11 tests d'interface				tous corrects, instantanés

Les 38 instances résolues dans le délai donnent toutes le statut attendu, avec des modèles valides et des algorithmes 1 et 2 concordants. Aucune réponse n'est fausse. Les 11 petits fichiers sont également testés à travers l'interface `sat.py`, dans les deux modes, exactement comme le fera le correcteur (fichier `tests/test_sat.py`). Les cas limites étaient déjà tous couverts par nos propres tests, ce qui confirme les choix du lecteur DIMACS (littéraux répétés, clause vide, formule sans clause).

Trois instances dépassent 60 s avec la graine 0. Pour savoir si la difficulté vient de l'instance ou du hasard, nous les avons relancées avec les graines 1, 2 et 3 (algorithme 2 seul) :

Instance	Graine 1	Graine 2	Graine 3
`public-12` (SAT)	1,1 s	2,5 s	timeout
`public-27` (SAT)	2,5 s	53,0 s	6,8 s
`public-03` (UNSAT)	timeout	timeout	timeout

Les deux instances satisfiables sont résolues en quelques secondes avec une graine favorable : comme sur Meowdoku, c'est la queue lourde du choix aléatoire. L'instance `public-03` est d'une autre nature : elle est insatisfiable et petite (63 variables, 168 clauses de 3 littéraux), mais aucune graine ne la résout. En 60 s, DPLL y prend environ 1,8 million de décisions, toutes suivies d'un retour arrière, sans terminer. Il s'agit vraisemblablement d'une des deux instances annoncées comme volontairement difficiles. Une heuristique ne suffira probablement pas : ce type d'instance motive des techniques qui dépassent DPLL (apprentissage de clauses). Nous y reviendrons à la question 6.

Interface en ligne de commande. Le fichier `sat.py` respecte dès maintenant l'interface imposée à la question 6 :

```
python sat.py instance.cnf --mode dpll --output-model
```

Le mode `dpll` utilise l'algorithme 2 ; le mode `optimized` l'utilise aussi en attendant la question 6. La sortie standard ne contient que la ligne `s SATISFIABLE` ou `s UNSATISFIABLE` et, sur demande, la ligne `v` du modèle complet, terminée par `0`. Un fichier illisible produit un message sur la sortie d'erreur et un code de retour non nul, sans rien écrire sur la sortie standard : une réponse incorrecte serait pénalisée en compétition. Des tests lancent le programme dans un processus séparé et vérifient exactement ce format.

5 Performances

5.1 Protocole

Le script `bench_q2.py` mesure le temps de résolution (`time.perf_counter`) et relève les compteurs de `Stats` : décisions, retours arrière, propagations unitaires et littéraux purs. Chaque résolution est enregistrée dans `results/q2_<expérience>.csv` ; les tableaux ci-dessous donnent des *médianes*, peu sensibles aux quelques exécutions extrêmes que le choix aléatoire produit. Une résolution est interrompue au bout de 30 s et comptée comme « timeout ». Les mesures ont été faites sur un Mac (Apple M1) avec Python 3.12 ; seuls les ordres de grandeur et les évolutions comptent, pas les valeurs absolues.

```
uv run python bench_q2.py --timeout 30
```

Nous utilisons cinq familles d'instances :

- des instances SATLIB, résolues par les deux algorithmes avec trois graines ;
- du 3-SAT aléatoire à $n = 40$ variables, en faisant varier le ratio m/n (nombre de clauses sur nombre de variables) ;
- du 3-SAT aléatoire au ratio 4,26, en faisant varier n ;
- des grilles Meowdoku générées par la question 1 ;
- une même instance résolue avec de nombreuses graines.

Le 3-SAT aléatoire suit le même modèle que SATLIB : chaque clause porte sur trois variables distinctes tirées au hasard, avec des signes aléatoires.

5.2 Algorithme 1 contre algorithme 2

Famille	Runs	Algo	Temps méd.	Temps max	Déc. méd.	Unit. méd.
`uf50-218`	150	1	0,020 s	0,098 s	133	1594
	150	2	0,020 s	0,099 s	133	1594
`uuf50-218`	150	1	0,072 s	0,131 s	461	5555
	150	2	0,072 s	0,131 s	461	5555
`uf75-325`	90	1	0,352 s	2,12 s	1336	21082
	90	2	0,349 s	2,13 s	1336	21082
`uuf75-325`	90	1	1,29 s	2,25 s	4696	73920
	90	2	1,32 s	2,29 s	4696	73920

Avec la même graine, les deux algorithmes font exactement les mêmes choix et parcourent le même arbre : tous les compteurs sont identiques. Les temps diffèrent de moins de 3 % : construire la valuation (une écriture dans un dictionnaire par variable affectée, sur la seule branche gagnante) est négligeable devant les simplifications. L'algorithme 2 ne coûte donc rien de plus et c'est lui que nous utilisons partout ailleurs.

5.3 Effet du ratio clauses / variables

m/n	SAT	UNSAT	Temps méd.	Décisions méd.	Unitaires méd.	Purs méd.
2	40	0	0,4 ms	7	4	26
3	40	0	0,5 ms	11	30	11
3,5	40	0	0,9 ms	16	90	11
4	32	8	4,8 ms	48	458	15
4,26	26	14	9,2 ms	86	853	15
4,5	22	18	12,3 ms	105	1080	11
5	3	37	15,4 ms	106	1104	3
6	0	40	10,6 ms	60	604	0
8	0	40	7,3 ms	26	255	0

($n = 40$, 40 formules par ratio.)

On retrouve le phénomène de *transition de phase* du 3-SAT aléatoire. Avec peu de clauses, presque toute valuation convient : DPLL trouve une solution quasiment sans retour arrière, et la règle du littéral pur est très utile (26 applications pour 7 décisions à $m/n = 2$). Avec beaucoup de clauses, la formule est très contrainte : la propagation unitaire produit vite une clause vide et l'insatisfiabilité est prouvée par un petit arbre. La difficulté est maximale entre les deux, là où la proportion d'instances satisfiables passe de 1 à 0, autour de $m/n \approx 4{,}26$ (une moitié environ de SAT). Pour $n = 40$, le pic de coût se situe un peu au-delà, vers 4,5–5 : à petite taille, la transition est étalée et les instances UNSAT proches du seuil, plus coûteuses, tirent la médiane vers la droite.

La règle du littéral pur devient inutile quand m/n augmente : chaque variable apparaît dans environ $3m/n$ clauses, et il est de moins en moins probable qu'elle n'apparaisse que sous une seule polarité.

5.4 Croissance avec le nombre de variables

n	Temps méd.	Temps max	Déc. méd.	Déc. SAT	Déc. UNSAT	ms / déc.
20	0,7 ms	2,0 ms	15	8	21	0,047
30	4,1 ms	8,2 ms	51	39	83	0,078
40	11 ms	50 ms	96	24	211	0,111
50	37 ms	84 ms	247	137	438	0,150
60	160 ms	563 ms	856	327	1570	0,185
70	635 ms	1447 ms	2632	1086	3780	0,235

(Ratio 4,26, 20 formules par taille, environ moitié SAT et moitié UNSAT.)

Croissance exponentielle. De $n = 20$ à $n = 70$, le nombre médian de décisions est multiplié par environ 175, soit un facteur $175^{1/50} \approx 1{,}11$ par variable supplémentaire : le nombre de décisions double environ toutes les 7 variables. C'est la croissance exponentielle attendue de DPLL sur des instances difficiles. À ce rythme, une instance aléatoire de 150 variables au seuil demanderait plus d'une heure.

Coût d'une décision. Le temps moyen par décision augmente aussi, de 0,05 à 0,24 ms, à peu près proportionnellement à n. C'est la conséquence directe de notre structure de données : chaque simplification recopie la formule, de taille proportionnelle à $m = 4{,}26\,n$, et une décision entraîne en moyenne une quinzaine de propagations unitaires, donc une quinzaine de copies. La complexité observée est bien (nombre de nœuds) $\times\, O(|F|)$.

SAT contre UNSAT. À taille égale, les instances UNSAT demandent 2 à 9 fois plus de décisions que les SAT. Pour prouver l'insatisfiabilité, DPLL doit réfuter *toutes* les branches, alors qu'il s'arrête dès la première solution d'une formule satisfiable. Sur les instances UNSAT, le nombre de retours arrière est d'ailleurs exactement égal au nombre de décisions : la première branche de chaque décision échoue forcément, sinon la formule serait satisfiable. C'est aussi pourquoi les temps des instances UNSAT sont plus réguliers (temps maximum proche de la médiane) : l'arbre complet dépend moins de la chance que la recherche d'une seule solution.

5.5 Instances Meowdoku

n	Var.	Résolues	Timeouts	Temps méd.	Temps max	Déc. méd.	Purs
4	16	10	0	0,2 ms	0,2 ms	5	0
6	36	10	0	0,7 ms	2,8 ms	6	0
8	64	10	0	1,8 ms	7,4 ms	10	0
10	100	10	0	4,5 ms	86 ms	13	0
12	144	10	0	17 ms	0,75 s	48	0
14	196	9	1	34 ms	26 s	177	0
16	256	6	4	1,3 s	7,8 s	6178	0

(10 grilles par taille, graine 0 ; médiane et maximum calculés sur les grilles résolues avant 30 s.)

Jusqu'à $n = 10$, les grilles sont résolues en quelques millisecondes : la structure du problème est très favorable à la propagation unitaire. Dès qu'un chat est placé, toutes les cases de sa ligne, de sa colonne, de sa région et ses voisines diagonales sont mises à faux par propagation, et une région réduite à une seule case libre force immédiatement son chat. Une dizaine de décisions suffit pour 100 variables, bien moins que pour du 3-SAT aléatoire de même taille.

La règle du littéral pur ne s'applique *jamais*, sur aucune grille. Cela s'explique par l'encodage. Une variable encore présente appartient à une région dont la clause positive n'est pas satisfaite (sinon la propagation l'aurait déjà mise à faux, par les clauses « au plus un chat par région ») : elle apparaît donc positivement. Elle apparaît aussi négativement dans les clauses binaires de sa ligne, de sa colonne ou de ses diagonales. Elle a donc les deux polarités, et n'est pas pure. Sur Meowdoku, la recherche de littéraux purs est un coût sans aucun bénéfice : c'est une piste directe d'optimisation pour la question 6.

À partir de $n = 12$, le comportement devient très irrégulier : la plupart des grilles restent faciles, mais certaines demandent des centaines de milliers de décisions. À $n = 16$, 4 grilles sur 10 dépassent 30 s. Le choix aléatoire n'exploite pas la structure du problème : il peut par exemple choisir de mettre à faux une case quelconque, ce qui ne propage presque rien, au lieu de placer un chat dans une région presque pleine.

5.6 Sensibilité au hasard

Instance	Graines	Temps min	Temps méd.	Temps max	Timeouts
`uf50-01` (SAT)	100	0,001 s	0,035 s	0,131 s	0
`uuf50-01` (UNSAT)	100	0,040 s	0,065 s	0,100 s	0
Meowdoku $n = 14$ (grille 2)	20	0,020 s	0,061 s	> 30 s	4

Une même formule peut être résolue 100 fois plus vite selon la graine lorsqu'elle est satisfiable : une suite de bons choix mène directement à une solution. Pour l'instance UNSAT, l'écart est bien plus faible (facteur 2,5), car l'arbre doit de toute façon être exploré en entier ; seul son nombre de nœuds dépend de l'ordre des choix. Le cas Meowdoku est le plus frappant : la grille qui avait dépassé le délai dans l'expérience précédente est résolue en 0,06 s pour la plupart des graines, mais 4 graines sur 20 dépassent encore 30 s. La distribution du temps a une « queue lourde » : un seul mauvais choix au début de la recherche peut coûter une exploration énorme.

5.7 Bilan

— Les deux algorithmes ont les mêmes performances ; l'algorithme 2 fournit le modèle sans surcoût mesurable.
— Le coût suit (nombre de décisions) $\times\, O(|F|)$: le nombre de décisions croît exponentiellement sur les instances difficiles, et chaque décision coûte une copie de la formule pour chaque simplification.
— Les instances UNSAT sont plus coûteuses et plus régulières que les instances SAT.
— Sur Meowdoku, la propagation unitaire fait l'essentiel du travail, la règle du littéral pur est inutile, et le choix aléatoire provoque de rares mais énormes explosions du temps de calcul à partir de $n \approx 14$.

Ces observations orientent la suite : des heuristiques de branchement (question 4) pour choisir des variables qui déclenchent beaucoup de propagation, puis une structure de données sans recopie et la suppression de la recherche de littéraux purs pour la version optimisée (question 6).

6 Usage des assistants IA